Matrices as Linear Maps

A matrix is a function that moves space

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

Matrices as transformations in neural networks

The trick a photo app runs a billion times a day

V VISUAL

original image (a grid of pixels)

every pixel × a rotation matrix

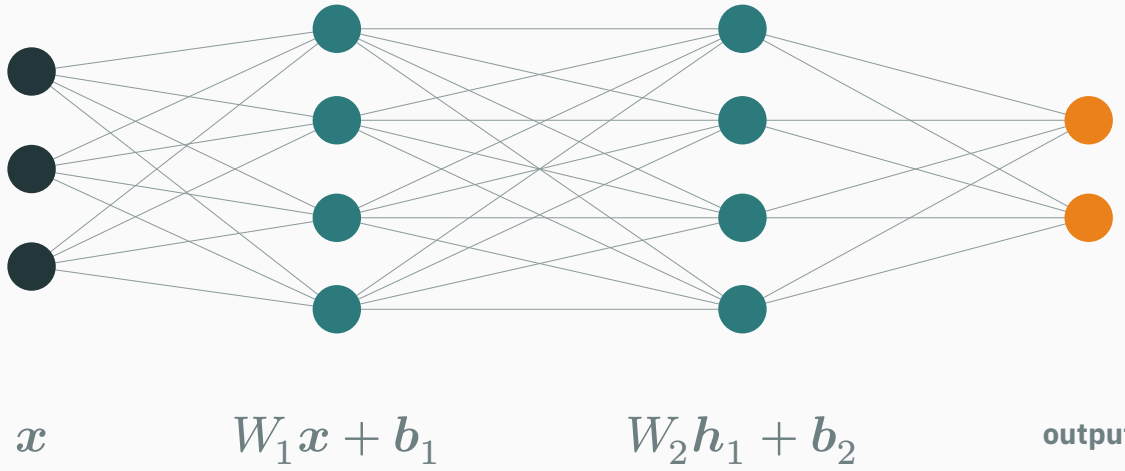
every pixel × a shear matrix

Rotating a photo touches a million pixels — yet the whole operation is **one 2×2 matrix**, applied to every pixel's coordinates.

Change four numbers → rotate becomes shear, zoom, mirror. The **matrix is the move**.

Inside every layer of every neural network

Every layer of every neural network computes the same thing: $\boldsymbol{h} = \varphi(W\boldsymbol{x} + \boldsymbol{b})$.



The $+\boldsymbol{b}$ merely slides space; φ bends it (a later course's story). The geometry — the part **training actually learns** — is $W\boldsymbol{x}$: **a matrix moving space**.

A GPT-class model is thousands of matrices applied in sequence. To understand the machine, first understand what one matrix does to space. That is today.

Learning outcomes

By the end of this lecture you will be able to:

1. Read a small matrix as a **move of space** — and sketch what it does to a grid.
2. Compute Ax two ways: the **row view** (dot products) and the **column view** (mix of columns).
3. Explain why matrix × matrix is **composition** of moves — and why $AB \neq BA$.
4. Compute **rank** on small matrices and say what it means geometrically: how much of space survives.
5. Decide when a matrix is **invertible** — and recognize when information is destroyed for good.
6. Solve $Ax = b$ the professional way (`solve`, never `inv`), and set up **least squares** for tall systems.

A matrix is a function

A second identity for an old object

So far a matrix has been a **table**: in L3, rows were data points (n students × d features).

Today the same object gets a second life. Feed a 2×2 matrix a vector — it hands back a vector:

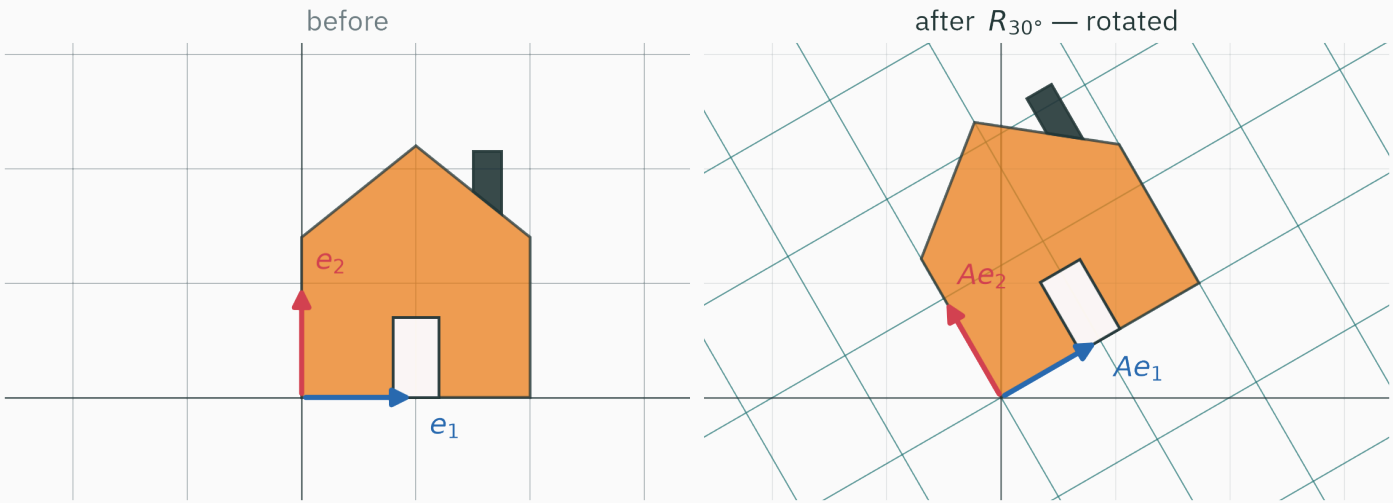
$$T(\boldsymbol{x}) = A\boldsymbol{x}, \qquad T : \mathbb{R}^2 \to \mathbb{R}^2$$

That makes A a **function**. Its input is every point of the plane; its output is where each point moves.

Same numbers, two readings — the table stores measurements; the function **does** something. Keep both in your head: ML constantly switches between them mid-equation.

Exhibit 1 · rotation

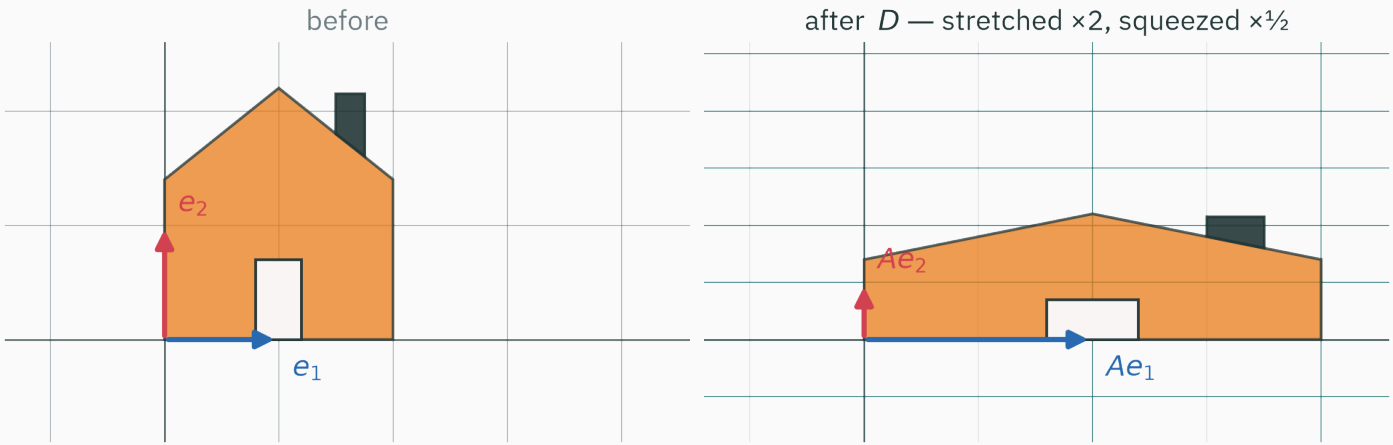
V VISUAL



$$R_{30°} = \begin{pmatrix} \cos 30° & -\sin 30° \\ \sin 30° & \cos 30° \end{pmatrix} \approx \begin{pmatrix} 0.87 & -0.50 \\ 0.50 & 0.87 \end{pmatrix}$$ — the whole plane turns; the house rides along

Exhibit 2 · stretch and squeeze

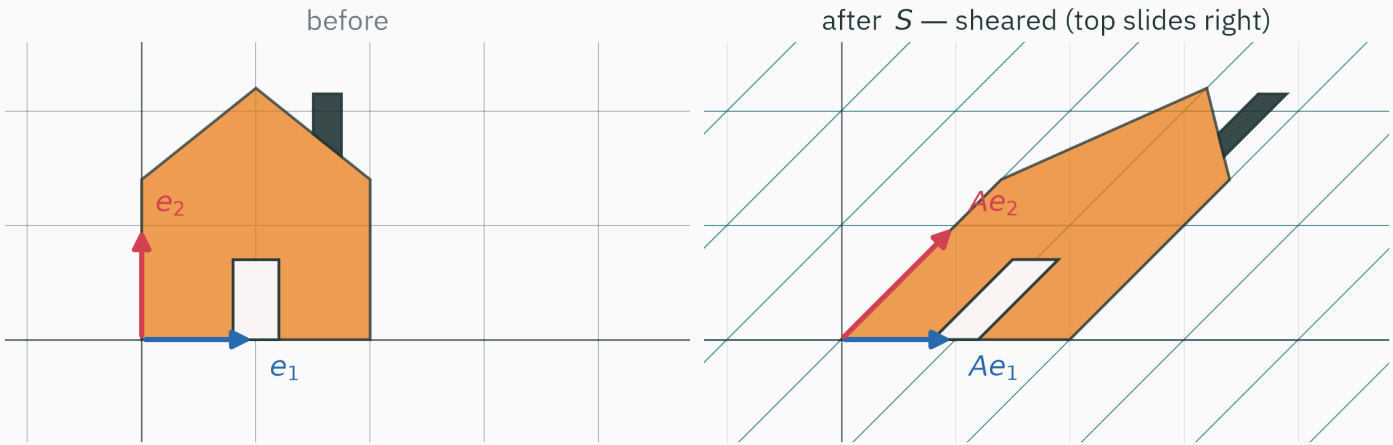
V VISUAL



$$D = \begin{pmatrix} 2 & 0 \\ 0 & 1/2 \end{pmatrix} \quad \text{— } \times 2 \text{ along } \; x \; , \times \; 1/2 \; \text{ along } \; y \; ; \text{ the grid squares become bricks}$$

Exhibit 3 · shear

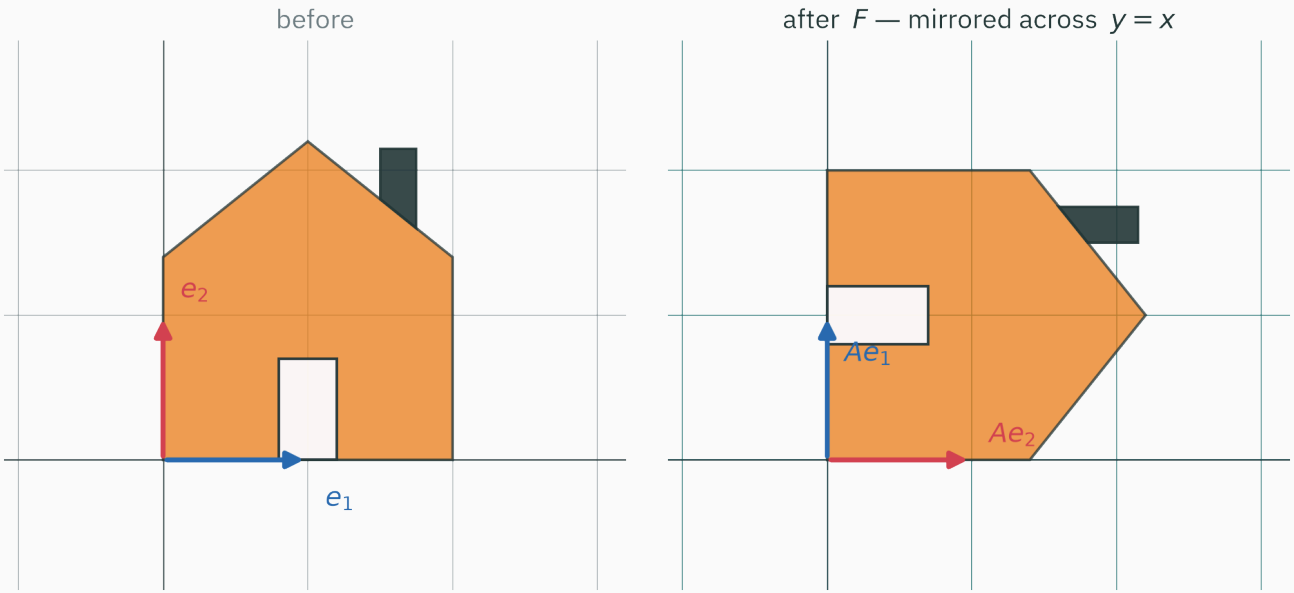
V VISUAL



$S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ — the ground floor holds still; every higher layer slides right

Exhibit 4 · reflection

V VISUAL



$$F = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \text{— mirror across the line } \; y = x$$

The chimney switched sides — no rotation can ever do that. Four matrices, four personalities.

What all four exhibits preserved

Look back at the after-grids. Every move kept three promises:

- straight lines stayed **straight** (nothing bent)
- parallel lines stayed **parallel**, evenly spaced — grid squares became identical tilted bricks
- the **origin stayed put**

Moves that keep these promises are called **linear**, and they obey two laws:

$$A(\boldsymbol{u}+\boldsymbol{v}) = A\boldsymbol{u} + A\boldsymbol{v}, \qquad A(c\boldsymbol{u}) = c(A\boldsymbol{u})$$

Linear = the move respects addition and scaling. Every claim in today's lecture flows from these two laws.

The mechanics · one entry at a time

How is Ax actually computed? Each output entry is a **row of A dotted with x**:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} ax_1 + bx_2 \\ cx_1 + dx_2 \end{pmatrix}$$

Check it on the shear, moving the point $(2, 1)$:

$$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 2+1 \\ 0+1 \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix} \quad \checkmark \text{ slid right, height kept}$$

L3 callback: each output entry is a **dot product** — row i asks the input "how much of you lies along me?" A neural layer with 4096 rows asks 4096 such questions at once.

The shape contract, in color

Matrices multiply vectors of **matching** shape — dimensions in orange must agree, and they are consumed:

$$\underbrace{\begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \end{pmatrix}}_{2\times 3} \underbrace{\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}}_{3\times 1} = \underbrace{\begin{pmatrix} 1\cdot 1 + 2\cdot 2 + 0\cdot 1 \\ 0\cdot 1 + 1\cdot 2 + 3\cdot 1 \end{pmatrix}}_{2\times 1} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$$

The general contract — inner dimensions kiss and cancel, outer dimensions survive:

$$(m \times n) \cdot (n \times 1) \to (m \times 1)$$

An $m \times n$ matrix eats vectors from $\mathbb{R}^n$ and returns vectors in $\mathbb{R}^m$ — it can move you between spaces.

Code · one @ moves a million points

```
import numpy as np
th = np.deg2rad(30)
R = np.array([[np.cos(th), -np.sin(th)],
              [np.sin(th),  np.cos(th)]])

R @ np.array([2.0, 1.0])        # array([1.232, 1.866]) — one point moved
```

```
pts = np.random.rand(1_000_000, 2)   # a million pixel coordinates
new = pts @ R.T                      # ...all rotated in one line
```

That second line **is** the photo app from the opening slide. No loops — the matrix moves all of space at once, and hardware loves it.

Symbols · what “linear map” means

A map $T : \mathbb{R}^n \to \mathbb{R}^m$ is **linear** if for all vectors $\boldsymbol{u}, \boldsymbol{v}$ and scalars c:

$$T(\boldsymbol{u} + \boldsymbol{v}) = T(\boldsymbol{u}) + T(\boldsymbol{v}), \qquad T(c\boldsymbol{u}) = c\,T(\boldsymbol{u})$$

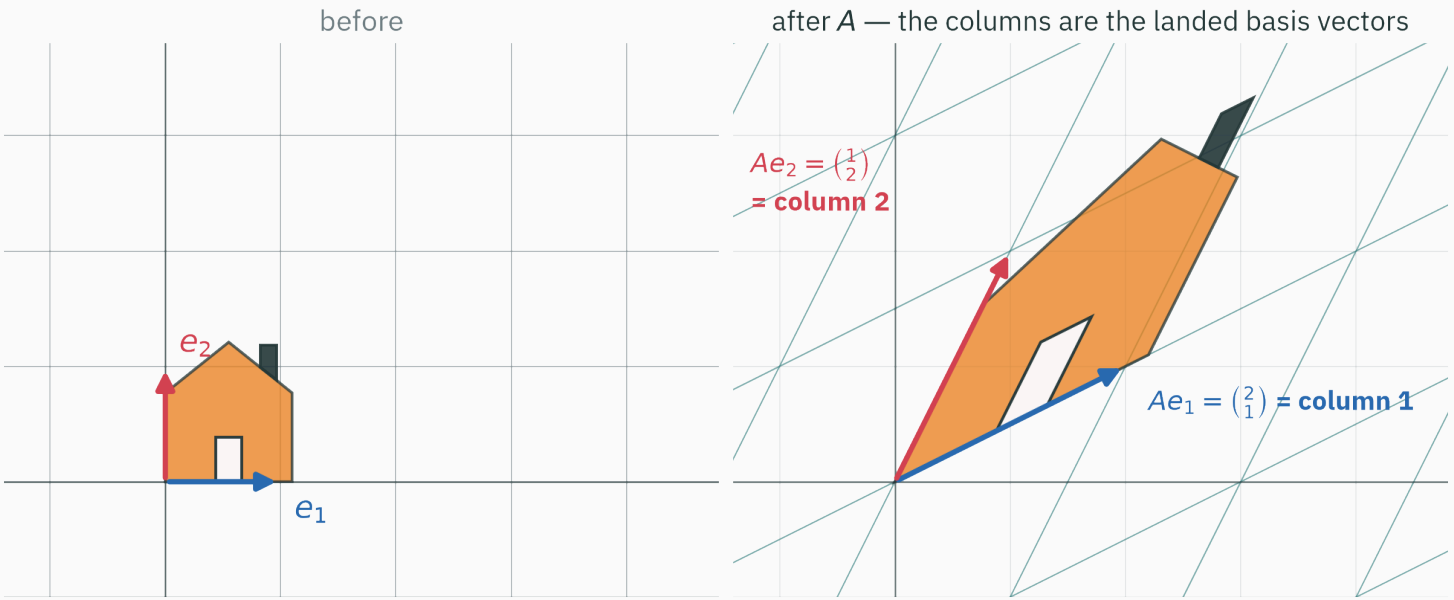
- Every matrix gives a linear map $\boldsymbol{x} \mapsto A\boldsymbol{x}$ — the two laws are exactly how the row-dot-product formula behaves.
- The converse holds too (stated, not proved): **every** linear map $\mathbb{R}^n \to \mathbb{R}^m$ is multiplication by some $m \times n$ matrix. There is nothing else.

“Matrix” and “linear move of space” are one concept in two costumes.

Read the columns

The trick · watch just two vectors

V VISUAL



You never need to track the whole plane — **two arrows carry the entire story**.

The columns are the landed basis

D DERIVATION

Where does $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ send the basis? Just multiply:

$$A\boldsymbol{e}_1 = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \text{column } 1, \qquad A\boldsymbol{e}_2 = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}\begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \text{column } 2$$

This is completely general — feeding in $\boldsymbol{e}_j$ picks out column j:

$$A\boldsymbol{e}_j = \text{the } \ j\text{-th column of } A$$

Sanity check with the do-nothing move: the identity $I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ has columns $\boldsymbol{e}_1, \boldsymbol{e}_2$ — "the basis lands exactly where it started." $I\boldsymbol{x} = \boldsymbol{x}$ for every $\boldsymbol{x}$.

The column picture of Ax

D DERIVATION

Why do two arrows determine everything? Any input is a mix of basis vectors:

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_1\boldsymbol{e}_1 + x_2\boldsymbol{e}_2$$

Now apply A and let linearity do all the work:

$$A\boldsymbol{x} = A(x_1\boldsymbol{e}_1 + x_2\boldsymbol{e}_2) = x_1(A\boldsymbol{e}_1) + x_2(A\boldsymbol{e}_2)$$

But $A\boldsymbol{e}_1, A\boldsymbol{e}_2$ are the **columns** $\boldsymbol{a}_1, \boldsymbol{a}_2$:

$$A\boldsymbol{x} = x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2$$

Ax is a recipe: mix the columns of A, using the entries of x as amounts. (Strang's column picture.)

Numbers · two readings, one answer

Compute $A\boldsymbol{x}$ for $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, $\boldsymbol{x} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ — both ways:

Row view — two dot products:

$$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \cdot 2 + 1 \cdot 1 \\ 1 \cdot 2 + 2 \cdot 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 4 \end{pmatrix}$$

Column view — a mix of columns, 2 parts and 1 part:

$$2\begin{pmatrix} 2 \\ 1 \end{pmatrix} + 1\begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \end{pmatrix} + \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 5 \\ 4 \end{pmatrix} \quad \checkmark \text{ same answer}$$

The row view is how machines compute. The column view is how **you should think** — it will explain rank, $A\boldsymbol{x} = \boldsymbol{b}$, and least squares within the hour.

Construct a matrix for a desired transformation

Want a matrix that rotates by $90°$? Don't memorize — **place the columns**:

e_1 must land at $(0, 1)$; e_2 at $(-1, 0)$. The landings **are** the columns:

$$R_{90°} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

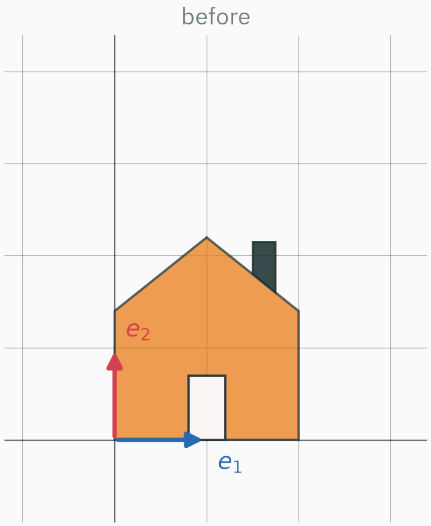
A "shadow" matrix that flattens everything onto the x-axis? e_1 stays, e_2 dies:

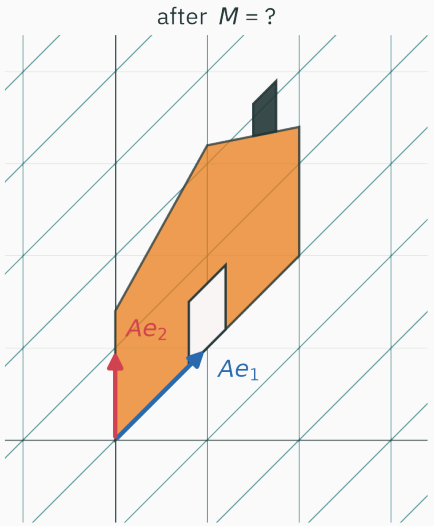
$$P = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

That second design **crushed a whole dimension on purpose**. Hold the thought — it returns in ten minutes with a name: rank 1.

Checkpoint: identify the transformation

Q QUESTION





Which matrix M made this move?

A. $M = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$

B. $M = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$

C. $M = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$

D. $M = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$

Answer · read the columns

A ANSWER

Correct: B — $M = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$

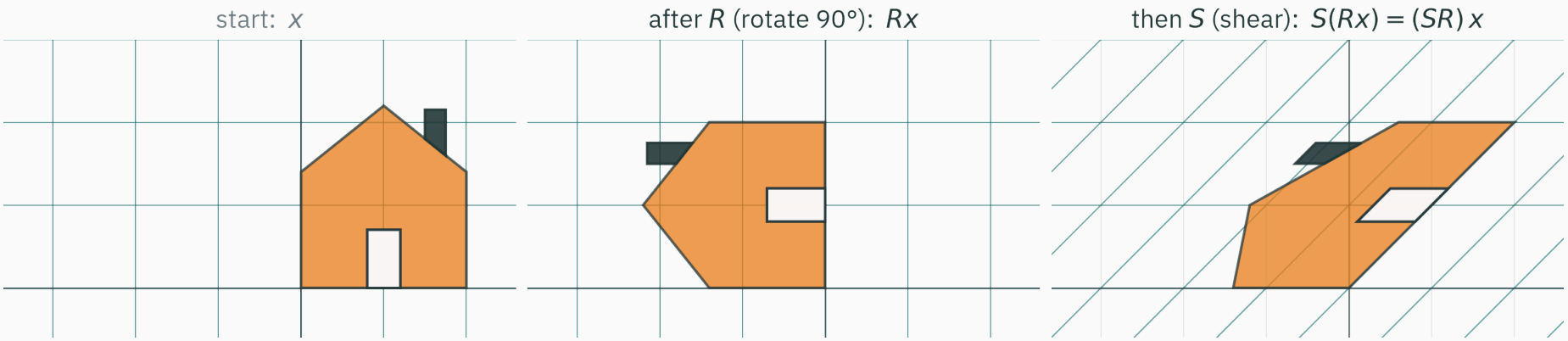
Why: Read the landings off the picture: e_1 landed at $(1, 1)$ — that must be column 1. e_2 stayed at $(0, 1)$ — that must be column 2. Only $\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ has those columns. (Option A is the **horizontal** shear from Exhibit 3 — it would leave e_1 untouched instead.)

Every “which matrix did this?” puzzle is solved the same way: **watch where the basis lands**.

Multiplication is composition

Two moves, one after the other

V VISUAL



Rotate with R, **then** shear with S: the input $\boldsymbol{x}$ becomes $S(R\boldsymbol{x})$.

Two linear moves in a row are still one linear move — so **some single matrix** must do the whole trip. We name it SR, written right-to-left: the move nearest $\boldsymbol{x}$ acts first.

The combined matrix, without any formula

Where does the **combined** move send the basis? Just follow each arrow through both moves:

$$(SR)\boldsymbol{e}_1 = S(R\boldsymbol{e}_1) = S\begin{pmatrix}0\\1\end{pmatrix} = \begin{pmatrix}1\\1\end{pmatrix}, \qquad (SR)\boldsymbol{e}_2 = S(R\boldsymbol{e}_2) = S\begin{pmatrix}-1\\0\end{pmatrix} = \begin{pmatrix}-1\\0\end{pmatrix}$$

The landings are the columns — the product is already done:

$$SR = \begin{pmatrix}1 & -1\\1 & 0\end{pmatrix}$$

To multiply matrices, push the columns of the right one through the left one: column j of AB is $A\boldsymbol{b}_j$.

The general recipe (and its shape)

$$(AB)_{ij} = \sum_k A_{ik} B_{kj} \quad \text{— row } i \text{ of } A \text{ dotted with column } j \text{ of } B$$

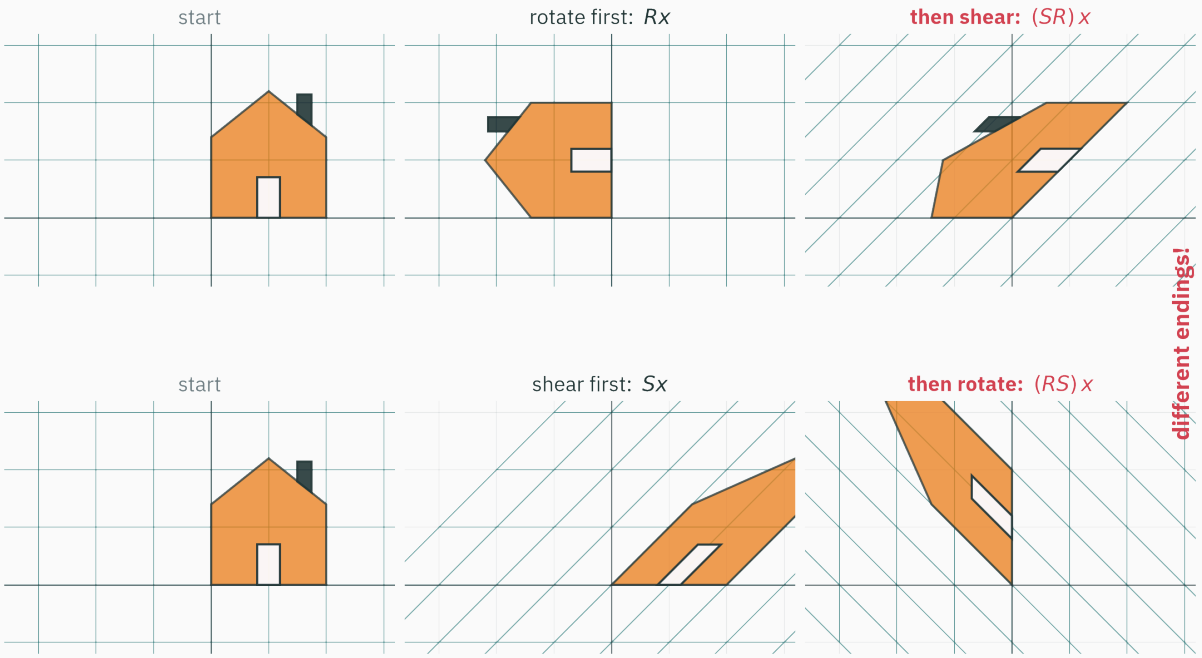
The shape contract again — inner dimensions consumed, outer survive:

$$(m \times n) \cdot (n \times p) \to (m \times p)$$

- The entry formula is the **bookkeeping** view — what hardware executes.
- Column-through view is the **thinking** view: AB = "A applied to each column of B" — a matmul is just p matvecs standing side by side.

Order matters

V VISUAL



Rotate-then-shear and shear-then-rotate end in **different places** — so $SR \neq RS$ before we compute a thing.

SR vs RS · the numbers agree with the picture

Track the basis both ways (or grind the entry formula — same result):

$$SR = \begin{pmatrix} 1 & -1 \\ 1 & 0 \end{pmatrix}, \qquad RS = \begin{pmatrix} 0 & -1 \\ 1 & 1 \end{pmatrix}$$

Different matrices, as the houses foretold.

$AB \neq BA$ in general. Matrix multiplication remembers the **order of moves** — "put on socks, then shoes" is not "put on shoes, then socks." Every identity you write this semester must respect it.

What **does** survive from ordinary algebra: associativity, $(AB)C = A(BC)$ — grouping is free, order is not.

Interactive · watch the machine run

I INTERACTIVE

INTERACTIVE

↗ http://matrixmultiplication.xyz/

Enter $S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ and $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ and watch the row-through-column machine grind out each entry — the row physically tips over and slides down the column. In class: predict every entry of RS **before** the animation lands.

Then feed it a $(2 \times 3)(3 \times 1)$ pair and watch the inner 3s consume each other — the shape contract, animated.

Code · `@` composes

```
S = np.array([[1., 1.], [0., 1.]])
R = np.array([[0., -1.], [1., 0.]])

S @ R                 # array([[ 1., -1.], [ 1.,  0.]])
R @ S                 # array([[ 0., -1.], [ 1.,  1.]])  — different!

x = np.array([2., 1.])
np.allclose((S @ R) @ x, S @ (R @ x))    # True — associativity
```

That last line is quietly profound: **precompute the combined move once**, or apply moves one at a time — identical result. Graphics engines and deep-learning compilers live on this choice.

Why stacked linear layers need a nonlinearity

Stack two purely linear layers, $\boldsymbol{h} = W_1 \boldsymbol{x}$ then $\boldsymbol{y} = W_2 \boldsymbol{h}$:

$$\boldsymbol{y} = W_2(W_1 \boldsymbol{x}) = (W_2 W_1)\boldsymbol{x} = W_{\text{eq}} \boldsymbol{x}$$

Composition collapses the stack: a 100-layer **linear** network is secretly **one matrix** — depth bought nothing.

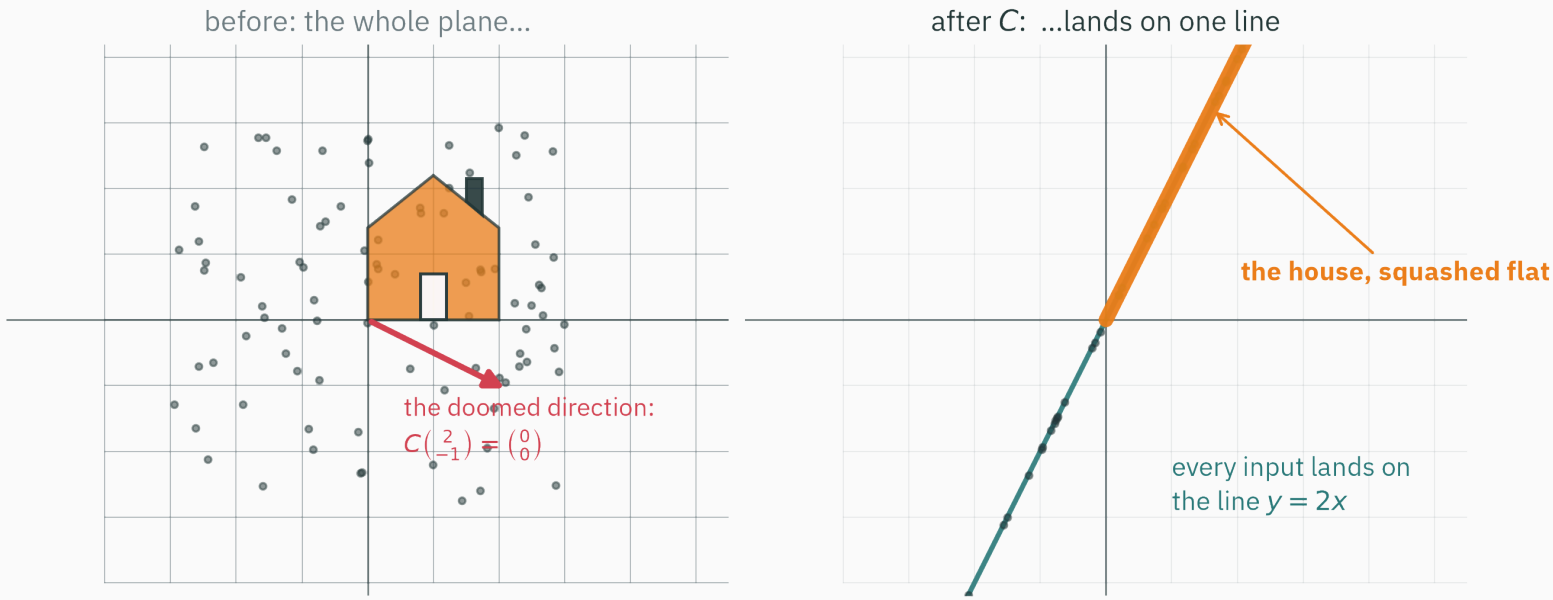
That is exactly why every real layer wraps $Wx + b$ in a nonlinear φ: it blocks the collapse, forcing each matrix to contribute a genuinely new move.

Move of space → bend → move of space → bend… — that alternation **is** deep learning. The moves are today's math; the bends belong to ES 667.

Rank: how much of space survives

Not every move preserves the plane

V VISUAL



$C = \begin{pmatrix}1 & 2\\ 2 & 4\end{pmatrix}$ takes the **entire 2-D plane** — houses, point clouds, everything — and lands it all on a single line. Something fundamental was lost.

The collapse, in numbers

Watch the columns of $C = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$: column 2 is exactly $2 \times$ column 1.

So every output — every mix $x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2$ — is a multiple of $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$:

$$C\boldsymbol{x} = x_1\begin{pmatrix} 1 \\ 2 \end{pmatrix} + x_2\begin{pmatrix} 2 \\ 4 \end{pmatrix} = (x_1 + 2x_2)\begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \text{— one direction, scaled. The line } \; y = 2x.$$

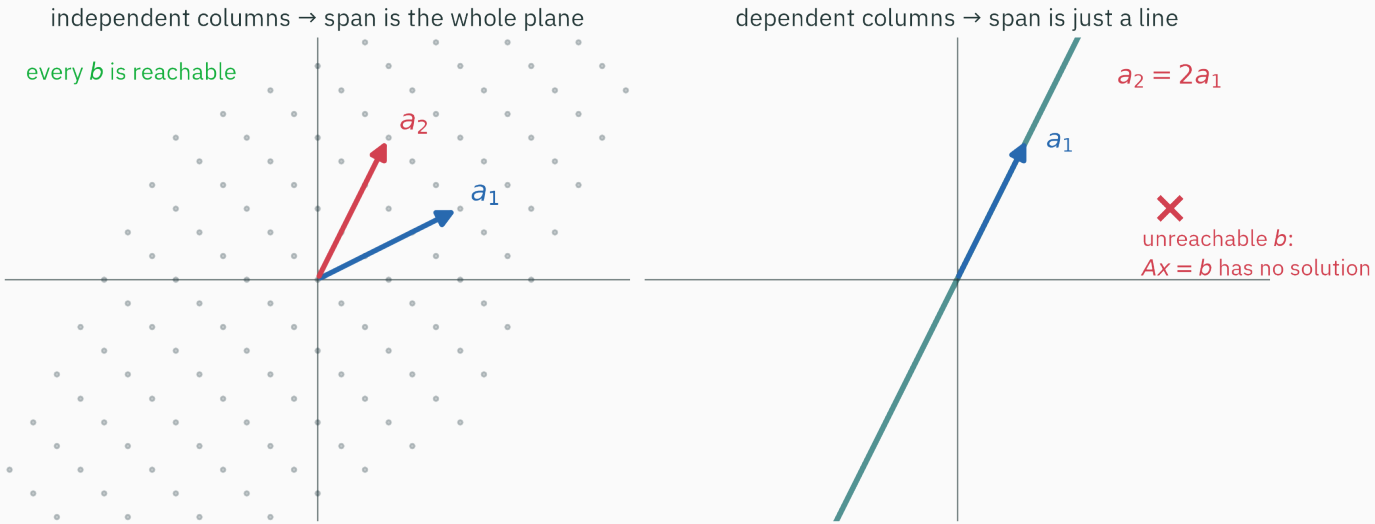
And one whole direction of inputs is **doomed**:

$$C\begin{pmatrix} 2 \\ -1 \end{pmatrix} = 2\begin{pmatrix} 1 \\ 2 \end{pmatrix} - 1\begin{pmatrix} 2 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

The columns fought over the same direction — and an entire dimension of information died in the crossfire.

What the columns can reach · column space

V VISUAL



$\mathrm{col}(A) = \mathrm{span}\{\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n\}$ — L3′s **span** of the columns: everything $A\boldsymbol{x}$ can ever reach.

Dependent columns shrink the span to a line, and most targets $\boldsymbol{b}$ become **unreachable** — exactly when $A\boldsymbol{x} = \boldsymbol{b}$ has no solution. (File that away; it returns in twenty minutes.)

Rank · the dimension of what survives

$$\text{rank}(A) = \text{number of linearly independent columns} = \dim(\text{col}(A))$$

Move	Matrix	Rank	What survives
rotation $R_{30°}$	$\begin{pmatrix} 0.87 & -0.5 \\ 0.5 & 0.87 \end{pmatrix}$	2	the whole plane
shear S	$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$	2	the whole plane
shadow P	$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$	1	the x-axis
collapse C	$\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$	1	the line $y = 2x$
zero matrix O	$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$	0	just the origin

For any $m \times n$ matrix: $\text{rank}(A) \leq \min(m, n)$ — you can't have more independent columns than columns, nor than the dimension they live in. Hitting the ceiling is called **full rank**.

The null space · what dies

The flip side of “what survives” is **which inputs get crushed to zero**:

$$\operatorname{null}(A) = \{\boldsymbol{x} : A\boldsymbol{x} = \mathbf{0}\} \qquad \text{for} \quad C : \text{the whole line through } \begin{pmatrix} 2 \\ -1 \end{pmatrix}$$

Dimensions are conserved — every input dimension either survives or dies:

$$\underbrace{\operatorname{rank}(A)}_{\text{survives}} + \underbrace{\dim \operatorname{null}(A)}_{\text{dies}} = \underbrace{n}_{\text{input dimensions}} \qquad \text{for} \quad C : \; 1 + 1 = 2 \; \checkmark$$

(The **rank–nullity theorem** — stated here, proved in MML 2.7.)

L5 plant: a direction that A merely **scales** — stretches without turning — is called an **eigenvector**. A doomed direction is the extreme case: an eigenvector with stretch factor 0. Next lecture is entirely about these special directions.

Rank by hand, one size up

$$A = \begin{pmatrix} 0 & 1 & 2 \\ 1 & 2 & 1 \\ 2 & 7 & 8 \end{pmatrix} \quad \text{— } 3 \times 3, \text{ so } \operatorname{rank} \leq 3. \text{ Hunt for a dependency.}$$

Row 3 is a mix of the others: $3 \cdot \text{row } 1 + 2 \cdot \text{row } 2 = (2, 7, 8) = \text{row } 3$. Only 2 independent rows → $\operatorname{rank}(A) = 2$.

> Row rank always equals column rank (MML 2.6), so test dependencies on whichever side is easier. Here the row dependency is immediate.

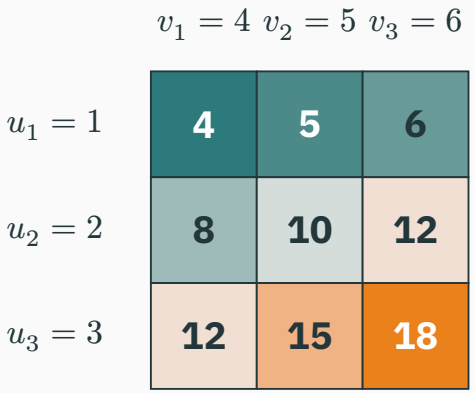
One more: $X = \begin{pmatrix} 1 & 2 & 4 & 4 \\ 3 & 4 & 8 & 0 \end{pmatrix}$ is 2×4, so $\operatorname{rank} \leq 2$; its two rows aren't proportional → $\operatorname{rank}(X) = 2$ — full rank, even though **some** columns repeat each other.

The thinnest matrices · outer products

D DERIVATION

Take $\boldsymbol{u} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$ and $\boldsymbol{v} = \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}$. Their **outer product** $\boldsymbol{u}\boldsymbol{v}^\top$ is a full 3×3 matrix:

$$\boldsymbol{u}\boldsymbol{v}^\top = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \begin{pmatrix} 4 & 5 & 6 \end{pmatrix} = \begin{pmatrix} 4 & 5 & 6 \\ 8 & 10 & 12 \\ 12 & 15 & 18 \end{pmatrix}$$



Cell (i, j) is simply $u_i v_j$ — a multiplication table.

Nine entries, but look at the shading: every column repeats the **same pattern**, just rescaled. This matrix is enormous bureaucracy around very little information.

Why $\operatorname{rank}(\boldsymbol{u}\boldsymbol{v}^\top) = 1$

D DERIVATION

Claim: for any nonzero $\boldsymbol{u}, \boldsymbol{v}$, the outer product $\boldsymbol{u}\boldsymbol{v}^\top$ has rank exactly 1.

Look at the columns. Column j of $\boldsymbol{u}\boldsymbol{v}^\top$ is $v_j\boldsymbol{u}$ — every column is a multiple of the **same** vector:

$$\text{col } 1 = 4\boldsymbol{u}, \quad \text{col } 2 = 5\boldsymbol{u}, \quad \text{col } 3 = 6\boldsymbol{u}$$

So the column space is just the line through $\boldsymbol{u}$: dimension 1 → rank 1. ■

Second look, via the action: $(\boldsymbol{u}\boldsymbol{v}^\top)\boldsymbol{x} = \boldsymbol{u}(\boldsymbol{v}^\top\boldsymbol{x})$ — a **number** times $\boldsymbol{u}$. Every input in $\mathbb{R}^3$ lands on $\boldsymbol{u}$'s line: the map measures you against $\boldsymbol{v}$, then points along $\boldsymbol{u}$.

An outer product $\boldsymbol{u}\boldsymbol{v}^\top$ is the thinnest nonzero matrix there is: rank 1.

Every matrix is a sum of rank-1 atoms

Read the claim backwards: rank-1 outer products are **atoms**, and every matrix is a molecule —

$$A = \sigma_1 \boldsymbol{u}_1 \boldsymbol{v}_1^\top + \sigma_2 \boldsymbol{u}_2 \boldsymbol{v}_2^\top + \cdots + \sigma_r \boldsymbol{u}_r \boldsymbol{v}_r^\top, \quad r = \operatorname{rank}(A)$$

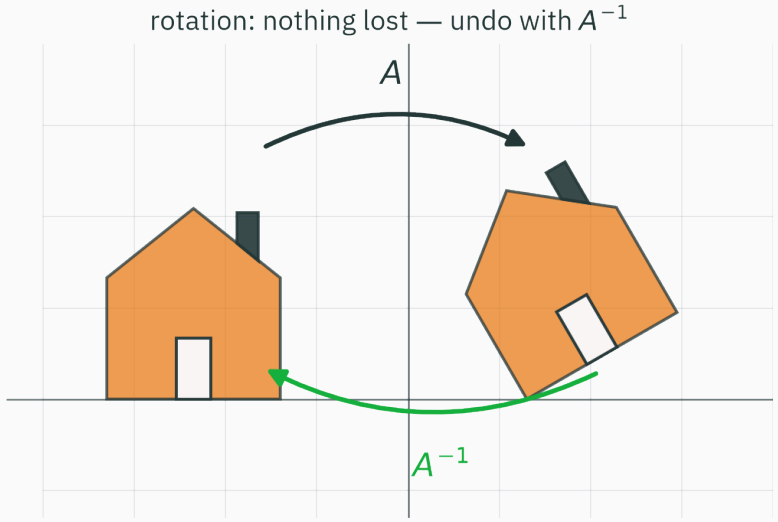
- L6 preview: the **SVD** finds the best atoms, importance-sorted $\sigma_1 \geq \sigma_2 \geq \ldots$ — keep the top few → compression, PCA.
- Modern fine-tuning (**LoRA**) freezes a giant W and learns only a low-rank update — a few outer products instead of billions of entries.

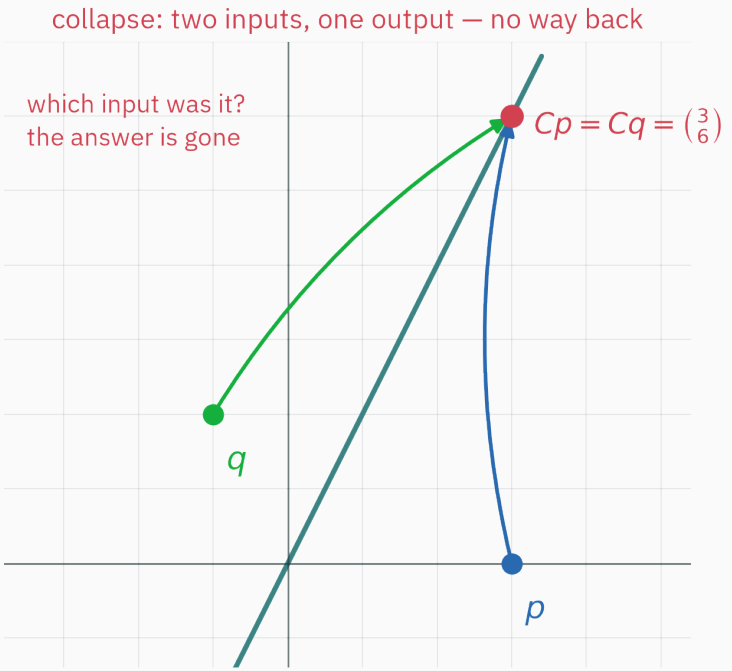
The statement "every column is a multiple of $\boldsymbol{u}$" is the rank-one structure used by low-rank parameter updates in L6.

Invertibility: undoing a move

Can you undo the move?

V VISUAL





The rotation is **reversible** — every landing point has exactly one origin story. The collapse is not: p and q land on the **same** output, and “which input was it?” has no answer.

The inverse · undo, in numbers

A^{-1} is the move that cancels A exactly — do, then undo, and nothing happened:

$$A^{-1}A = AA^{-1} = I$$

For our exhibits the undo is guessable from the **geometry**, no formula needed:

$$R_{30^\circ}^{-1} = R_{-30^\circ} \quad \text{(rotate back)}, \qquad S^{-1} = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix} \quad \text{(slide the top back left)}$$

Verify the shear by composing:

$$\begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \quad \checkmark$$

When an inverse exists, it is **unique** — there is only one way to perfectly undo a move.

When can you undo? One test in three costumes

For a **square** matrix A, these are the same single question:

- **full rank** — columns independent: the outputs fill all of $\mathbb{R}^n$, nothing was flattened
- **null space is only $\{0\}$** — no nonzero direction is removed, so distinct inputs do not collide
- $\det(A) \neq 0$ — the determinant measures the **area scale factor** of the move; $\det = 0$ means areas got crushed flat (we'll meet $\det$ properly in L5)

A matrix passing the test is **invertible** (nonsingular); failing, **singular**.

Invertible $\Leftrightarrow$ full rank $\Leftrightarrow$ nothing is crushed. You can undo a move exactly when it destroyed no information.

Checkpoint · spot the un-undoable

Q QUESTION

Without computing a single determinant: which matrix has no inverse? (Hint: one of them flattens the plane.)

A. $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$

B. $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$

C. $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$

D. $\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$

Answer · spot the un-undoable

A ANSWER

Correct: B — $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$ is singular

Why: Its second column is $2\times$ the first — dependent columns → rank 1 → the plane is flattened onto a line → two different inputs share each output, so no undo can exist. The rotation (A), shear (C), and scale (D) all keep rank 2: nothing dies, so each can be undone.

Notice you **never** needed arithmetic — reading the columns answered a determinant question.

Solving Ax = b

The most-solved equation in computing

Flip the question. So far: given x, where does it land? Now **the landing b is known; the input is not**:

$$A\boldsymbol{x} = \boldsymbol{b} \quad \text{— which input lands on } \boldsymbol{b}?$$

The column picture makes it concrete — **find the recipe** mixing the columns into $\boldsymbol{b}$:

$$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \boldsymbol{x} = \begin{pmatrix} 4 \\ 5 \end{pmatrix} \quad \Rightarrow \quad \boldsymbol{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}: \quad 1\begin{pmatrix} 2 \\ 1 \end{pmatrix} + 2\begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 4 \\ 5 \end{pmatrix} \quad \checkmark$$

Solvable for every $\boldsymbol{b}$, uniquely, exactly when A is invertible — nothing crushed. By hand for two unknowns; but one weather step or network layer has $n \approx 10^6$. We need the machine — **carefully**.

On paper, $x = A^{-1}b$ · on silicon, never

$x = A^{-1}b$ is a theorem, **not an algorithm**. Watch both routes on a notoriously nasty (but tiny!) matrix — the 12×12 Hilbert matrix of simple fractions $H_{ij} = 1/(i + j + 1)$, with true solution all-ones:

```
n = 12; i = np.arange(n)
H = 1.0 / (i[:, None] + i[None, :] + 1)      # Hilbert matrix
x_true = np.ones(n);  b = H @ x_true

np.abs(np.linalg.inv(H) @ b - x_true).max()   # 8.66    ← inv route
np.abs(np.linalg.solve(H, b) - x_true).max()  # 0.143   ← solve route
```

The true entries are all 1. The `inv` route is off by **8.66** — a 60× larger error than `solve`, on a matrix that fits on this slide.

Why `solve` wins

- **Fewer operations** — `inv` secretly solves n systems (one per column of I), then still multiplies: measured $\approx 2.3\times$ slower at $n = 2000$, and every extra operation is an extra rounding.
- **One rounding cascade, not two** — L2 callback: each float op tells a tiny lie. `solve` factors A once (LU factorization — L17 opens the box) and back-substitutes; `inv` commits its lies into an explicit A^{-1}, then compounds them in the multiply.

Write `np.linalg.solve(A, b)`. An `inv(A) @ b` in numerical code is a code-review flag across the entire industry — same theorem, worse arithmetic.

Honest footnote: $\mathrm{cond}(H) \approx 10^{16}$, so even `solve` sweated here (error 0.14). Why some matrices **amplify** rounding lies — the condition number — is Lecture 17's opening act.

Too many equations · tall systems

Fit a line $\hat{y} = a + bt$ through **three** points $(0, 1), (1, 3), (2, 4)$ — three equations, two unknowns:

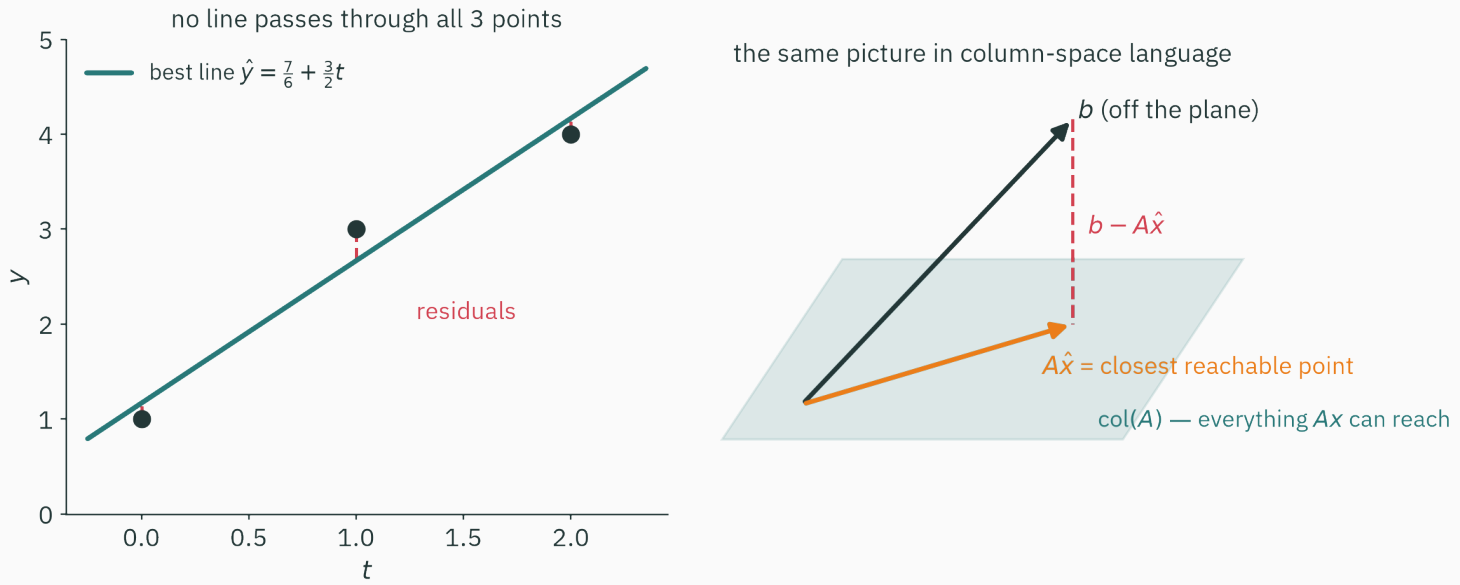
$$\underbrace{\begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}}_{3\times 2 \text{ — tall}} \begin{pmatrix} a \\ b \end{pmatrix} = \underbrace{\begin{pmatrix} 1 \\ 3 \\ 4 \end{pmatrix}}_{\boldsymbol{b}\in\mathbb{R}^3}$$

Now the column picture delivers bad news: $\mathrm{col}(A)$ — everything the two columns can mix — is only a **plane** inside $\mathbb{R}^3$, and our $\boldsymbol{b}$ is off it.

A tall system generically has no exact solution: three demands, two dials.

No line fits · the miss, drawn

V VISUAL



Left: no line threads all three points — every candidate leaves **residuals**. Right: the same fact in column-space language — b hovers off the reachable plane.

Least squares · aim for the shadow

If you can't reach $\boldsymbol{b}$, reach the **closest point you can** — minimize the miss:

$$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}} \|A\boldsymbol{x} - \boldsymbol{b}\|^2$$

Geometrically (right panel, a moment ago): $A\hat{\boldsymbol{x}}$ is the **projection** — L3's shadow! — of $\boldsymbol{b}$ onto $\mathrm{col}(A)$; the leftover error sticks out perpendicular.

```
A = np.array([[1., 0.], [1., 1.], [1., 2.]])
b = np.array([1., 3., 4.])
np.linalg.lstsq(A, b, rcond=None)[0]   # (7/6, 3/2): best line 7/6 + 1.5·t
```

Two IOUs: **L6** computes least squares at scale (via SVD), and **L14** reveals why **squared** error is the statistically right miss to minimize. Fitting = projection is the bridge.

Lecture 4 — summary

- **A matrix is a function**: $\boldsymbol{x} \mapsto A\boldsymbol{x}$ moves all of space — lines stay straight, parallels parallel, origin fixed.
- **Columns = landed basis**: $A\boldsymbol{e}_j = \boldsymbol{a}_j$, so $A\boldsymbol{x} = x_1\boldsymbol{a}_1 + \ldots + x_n\boldsymbol{a}_n$ — a recipe mixing columns.
- **Multiplication is composition**: column j of AB is $A\boldsymbol{b}_j$; order matters, grouping doesn't.
- **Rank** = dimension of what survives; **null space** = what dies; $\text{rank} + \dim \text{null} = n$; $\boldsymbol{u}\boldsymbol{v}^\top$ = the rank-1 atom.
- **Invertible ⇔ full rank**: undo exists iff no information was destroyed.
- **`solve`, never `inv`**, for $A\boldsymbol{x} = \boldsymbol{b}$ — fewer roundings, faster (conditioning: L17).
- **Tall systems**: no exact solution — least squares projects $\boldsymbol{b}$ onto $\text{col}(A)$ (L6, L14).

Read before L5 — MML Ch 2.5–2.8. **Tutorial 2** this week: the transformation gallery, rank experiments, and the Hilbert `solve`-vs-`inv` shoot-out, in NumPy.

Practice problems · I

Try on paper; verify in the Tutorial 2 notebook.

1. Design by columns: the matrix reflecting across the x-axis, and the one rotating by $180°$. Check by tracking $\boldsymbol{e}_1, \boldsymbol{e}_2$.
2. Compute $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ — one swaps columns, one swaps rows. Which is which, and why?
3. Ranks of $\begin{pmatrix} 1 & 2 & 4 & 4 \\ 3 & 4 & 8 & 0 \end{pmatrix}$ and $\begin{pmatrix} 5 & 4 & 3 \\ 4 & 3 & 2 \\ 3 & 2 & 1 \end{pmatrix}$? Hunt for dependencies before trusting your gut.

Practice problems · II

4. For $\boldsymbol{u} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\boldsymbol{v} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$: write $\boldsymbol{u}\boldsymbol{v}^\top$ and find the direction it kills. Hint: what is perpendicular to $\boldsymbol{v}$?
5. Socks and shoes: why is $(AB)^{-1} = B^{-1}A^{-1}$? Think "undo two moves in the right order" — then verify on SR.
6. On the $n = 8$ Hilbert matrix, predict the more accurate of `solve` and `inv @ b`; run both. At what n does `solve` itself lose digits?

A matrix is a verb, not a table.

Next: **Eigendecomposition** — the directions a matrix cannot turn, and how Google used one matrix, applied over and over, to rank the entire web.